



Chapter 5. Managing Projects with Poetry

The preceding chapters introduced the building blocks for publishing production-quality Python packages. So far, you’ve written a *pyproject.toml* for a project; created an environment and installed dependencies with *uv*, *pip*, or *pip-tools*; and built and published packages with *build* and *Twine*.

By standardizing project metadata and build backends, *pyproject.toml* broke the *setuptools* monopoly (see [“The Evolution of Python Project Managers”](#)) and brought diversity to the packaging ecosystem. Defining a Python package got easier, too: a single well-specified file with great tooling support replaces the legacy boilerplate of *setup.py* and untold configuration files.

Yet, some problems remain.

Before you can work on a *pyproject.toml*-based project, you need to research packaging workflows, configuration files, and associated tooling. You have to choose one of a number of available build backends ([Table 3-2](#))—and many people don’t know what those are, let alone how to choose them. Important aspects of Python packages remain unspecified—for example, how project sources are laid out and which files should go into the packaging artifacts.

Dependency and environment management could be easier, too. You need to handcraft your dependency specifications and compile them with *pip-tools*, cluttering your project with requirements files. And it can be hard to keep track of the many Python environments on a typical developer system.

The Python project manager Poetry was addressing these problems before some of the standards governing *pyproject.toml* took shape. Its friendly command-line interface lets you perform most tasks related to packaging, dependencies, and environments. Poetry brings its own standards-compliant build backend, `poetry.core`—but you can remain blissfully unaware of this fact. It also comes with a strict dependency resolver and locks all dependencies by default, behind the scenes.

Why learn about packaging standards and low-level plumbing if Poetry abstracts away many of these details? Because, while Poetry ventures into new territory, it still works within the framework defined by packaging standards. Mechanisms like dependency specifications and virtual environments power its central features. Interoperability standards let Poetry interact with package repositories as well as other build backends and package installers.

An understanding of these underlying mechanisms helps you debug situations where Poetry's convenient abstractions break down—for example, when a misconfiguration or a bug causes a package to end up in the wrong environment. Finally, the experience of past decades teaches us that tools come and go, while standards and algorithms are here to stay.

A decade ago, Python packaging was firmly in the hands of three tools: `setuptools`, `virtualenv`, and `pip`. You'd use `setuptools` to create Python packages, `virtualenv` to set up virtual environments, and `pip` to install packages into them. Everybody did. Around 2016—the same year that the *pyproject.toml* file became standard—things started to change.

In 2015, Thomas Kluyver began developing `Flit`, an alternative build tool that could create packages and publish them to PyPI. In 2016, Donald Stufft from the `pip` maintainer team started working on `Pipfile`, a proposed replacement for requirements files, including a specification of lock files. In 2017, his work led to Kenneth Reitz's `Pipenv`, which allows you to manage dependencies and environments for Python applications and deploy them in a reproducible way. `Pipenv` deliberately didn't package your application: you'd just keep a bunch of Python modules in a Git repository.

`Poetry`, started in 2018 by Sébastien Eustace, was the first tool to provide a unified approach to packaging, dependencies, and environments—and quickly became widely adopted. Two other tools follow a similarly holistic approach: `PDM`, started by Frost Ming in 2019, and `Hatch` by Ofek Lev in 2017. `Hatch` has recently grown in popularity, especially among tooling and library developers. In 2023, they were joined by `Rye`, a project manager written in Rust by Armin Ronacher. In addition, `Hatch` and `Rye` also manage Python installations, leveraging the Python Standalone Builds project.

`Poetry`, `Hatch`, `PDM`, and `Rye` each give you an integrated workflow for managing Python packages, environments, and dependencies. As such, they've come to be known as *Python project managers*. Keep an eye on Astral's `uv` as well!

Installing Poetry

Install Poetry globally using `pipx` to keep its dependencies isolated from the rest of the system:

```
$ pipx install poetry
```

A single Poetry installation works with multiple Python versions. However, Poetry uses its own interpreter as the default Python version. For this reason, it's worthwhile to install Poetry on the latest stable Python release. When installing a new feature release of Python, reinstall Poetry like this:

```
$ pipx reinstall --python=3.12 poetry
```

You can omit the `--python` option if `pipx` already uses the new Python version (see [“Configuring pipx”](#)).

When a prerelease of Poetry becomes available, you can install it side by side with the stable version:

```
$ pipx install poetry --suffix=@preview --pip-args=--pre
```

Here, I've used the `--suffix` option to rename the command so you can invoke it as `poetry@preview` while keeping `poetry` as the stable version. The `--pip-args` option lets you pass options to `pip`, like `--pre` for including prereleases.

NOTE

Poetry also comes with an [official installer](https://install.python-poetry.org), which you can download and run with Python. It's not as flexible as `pipx`, but it provides a readily available alternative:

```
$ curl -sSL https://install.python-poetry.org | python3 -
```

Upgrade Poetry periodically to receive improvements and bugfixes:

```
$ pipx upgrade poetry
```

Type `poetry` on its own to check your installation of Poetry. Poetry prints its version and usage to the terminal, including a useful listing of all available subcommands:

```
$ poetry
```

Having successfully installed Poetry, you may want to enable tab completion for your shell. Use the command `poetry help completions` for shell-specific instructions. For example, the following commands enable tab completion in the Bash shell:

```
$ poetry completions bash >> ~/.bash_completion  
$ echo ". ~/.bash_completion" >> ~/.bashrc
```

Restart your shell for the changes to take effect.

Creating a Project

You can create a new project using the command `poetry new`. As an example, I'll use the `random-wikipedia-article` project from previous chapters. Run the following command in the parent directory where you want to keep your new project:

```
$ poetry new --src random-wikipedia-article
```

After running this command, you'll see that Poetry created a project directory named *random-wikipedia-article*, with the following structure:

```
random-wikipedia-article
├── README.md
├── pyproject.toml
├── src
│   ├── random_wikipedia_article
│   │   └── __init__.py
└── tests
    └── __init__.py
```

The `--src` option instructs Poetry to place the import package in a subdirectory named *src* rather than directly in the project directory.

THE SRC LAYOUT

Until a few years ago, package authors placed the import package directly in the project directory. These days, a project layout with *src*, *tests*, and *docs* directories at the top is becoming more common.

Keeping the import package tucked away under *src* has practical advantages. During development, the current directory often appears at the start of `sys.path`. Without an *src* layout, you may be importing your project from its source code, not from the package you've installed in the project environment. In the worst case, your tests could fail to detect issues in a release you're about to publish.

On the other hand, whenever you *want* to execute the source code itself, editable installs achieve this by design. With an *src* layout, packaging tools can implement editable installs by adding the *src* directory to `sys.path`—without the side effect of making unrelated Python files importable.

Let's take a look at the generated *pyproject.toml* ([Example 5-1](#)).

Example 5-1. A *pyproject.toml* file for Poetry

```
[tool.poetry]
name = "random-wikipedia-article"
version = "0.1.0"
description = ""
authors = ["Your Name <you@example.com>"]
readme = "README.md"
packages = [{include = "random_wikipedia_article", from = "src"}]

[tool.poetry.dependencies]
python = "^3.12"

[build-system]
requires = ["poetry-core"]
build-backend = "poetry.core.masonry.api"
```

Poetry has created a standard `build-system` table with its build backend, `poetry.core`. This means anybody can install your project from source using `pip` or `uv`—no need to set up, or even know about, the Poetry project manager. Similarly, you can build packages using any standard build frontend, such as `build`:

```
$ pipx run build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - poetry-core
* Getting build dependencies for sdist...
* Building sdist...
* Building wheel from sdist
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - poetry-core
* Getting build dependencies for wheel...
* Building wheel...
Successfully built random_wikipedia_article-0.1.0.tar.gz
and random_wikipedia_article-0.1.0-py3-none-any.whl
```

The Project Metadata

You may be surprised to see the project metadata appear under `tool.poetry` instead of the familiar `project` table (see [“Project Metadata”](#)). The Poetry project plans to support the project metadata standard in its next major release.¹ As you can see in [Table 5-1](#), most fields have the same name and a similar syntax and meaning.

[Example 5-2](#) fills in the metadata for the project. I’ve highlighted some differences from [Example 3-4](#). (You’ll use the command-line interface to add the dependencies later.)

Example 5-2. Metadata for a Poetry project

```
[tool.poetry]
name = "random-wikipedia-article"
version = "0.1.0"
description = "Display extracts from random Wikipedia articles"
keywords = ["wikipedia"]
license = "MIT" ❶
classifiers = [
    "License :: OSI Approved :: MIT License",
    "Development Status :: 3 - Alpha",
    "Environment :: Console",
    "Topic :: Games/Entertainment :: Fortune Cookies",
]
authors = ["Your Name <you@example.com>"] ❷
readme = "README.md" ❸
homepage = "https://yourname.dev/projects/random-wikipedia-article" ❹
repository = "https://github.com/yourname/random-wikipedia-article"
documentation = "https://readthedocs.io/random-wikipedia-article"
packages = [{include = "random_wikipedia_article", from = "src"}]

[tool.poetry.dependencies]
python = ">=3.10" ❺

[tool.poetry.urls]
Issues = "https://github.com/yourname/random-wikipedia-article/issues"
```



```
[tool.poetry.scripts]
```

```
random-wikipedia-article = "random_wikipedia_article:main"
```

- ❶ The `license` field is a string with a SPDX identifier, not a table.
- ❷ The `authors` field contains strings in the format `"name <email>"`, not tables. Poetry prepopulates the field with your name and email from Git.
- ❸ The `readme` field is a string with the file path. You can also specify multiple files as an array of strings, such as *README.md* and *CHANGELOG.md*. Poetry concatenates them with a blank line in between.
- ❹ Poetry has dedicated fields for some project URLs, namely its homepage, repository, and documentation; for other URLs, there's also a generic `urls` table.
- ❺ The `python` entry in `dependencies` lets you declare compatible Python versions. For this project, you require Python 3.10 or later.

Table 5-1. Metadata fields in `tool.poetry`

Field	Type	Description	project field
<code>name</code>	String	The project name	<code>name</code>
<code>version</code>	String	The version of the project	<code>version</code>
<code>description</code>	String	A short description of the project	<code>description</code>
<code>keywords</code>	Array of strings	A list of keywords for the project	<code>keywords</code>
<code>readme</code>	String or array of strings	A file or list of files with the project description	<code>readme</code>
<code>license</code>	String	A SPDX license identifier, or “Proprietary”	<code>license</code>
<code>authors</code>	Array of strings	The list of authors	<code>authors</code>
<code>maintainers</code>	Array of strings	The list of maintainers	<code>maintainers</code>
<code>classifiers</code>	Array of strings	A list of classifiers describing the project	<code>classifiers</code>
<code>homepage</code>	String	The URL of the project homepage	<code>urls</code>
<code>repository</code>	String	The URL of the project repository	<code>urls</code>
<code>documentation</code>	String	The URL of the project documentation	<code>urls</code>
<code>urls</code>	Table of strings	The project URLs	<code>urls</code>
<code>dependencies</code>	Table of strings or tables	Required third-party packages	<code>dependencies</code>
<code>extras</code>	Table of tables	Optional third-party packages	<code>optional-dependencies</code>

Field	Type	Description	project field
groups	Table of tables	Dependency groups	<i>none</i>
scripts	Table of strings or tables	Entry-point scripts	scripts
plugins	Table of tables of strings	Entry-point groups	entry-points

Some `project` fields have no direct equivalent under `tool.poetry`:

- There's no `requires-python` field; instead, you specify the required Python version in the `dependencies` table, using the `python` key.
- There's no dedicated field for GUI scripts; use `plugins.gui_scripts` instead.
- There's no `dynamic` field—all metadata is Poetry-specific, so declaring dynamic fields wouldn't make much sense.

Before we move on, let's check that the *pyproject.toml* file is valid. Poetry provides a convenient command to validate the TOML file against its configuration schema:

```
$ poetry check
All set!
```

The Package Contents

Poetry allows you to specify which files and directories to include in the distribution—a feature still missing from the *pyproject.toml* standards ([Table 5-2](#)).

Table 5-2. Package content fields in `tool.poetry`

Field	Type	Description
<code>packages</code>	Array of tables	Patterns for modules to include in the distribution
<code>include</code>	Array of strings or tables	Patterns for files to include in the distribution
<code>exclude</code>	Array of strings or tables	Patterns for files to exclude from the distribution

Each table under `packages` has an `include` key with a file or directory. You can use `*` and `**` wildcards in their names and paths, respectively. The `from` key allows you to include modules from subdirectories such as `src`. Finally, you can use the `format` key to restrict modules to a specific distribution format; valid values are `sdist` and `wheel`.

The `include` and `exclude` fields allow you to list other files to include in, or exclude from, the distribution. Poetry seeds the `exclude` field using the `.gitignore` file, if present. By default, Poetry includes these additional files in source distributions only. Instead of a string, you can use a table with `path` and `format` keys to specify the distribution formats that should include the files. [Example 5-3](#) shows how to include the test suite in source distributions.

Example 5-3. Including the test suite in source distributions

```
packages = [{include = "random_wikipedia_article", from = "src"}]  
include = ["tests"]
```

The Source Code

Copy the contents of [Example 5-4](#) into the `__init__.py` file in the new project.

Example 5-4. The source code for random-wikipedia-article

```
import httpx
from rich.console import Console

from importlib.metadata import metadata

API_URL = "https://en.wikipedia.org/api/rest_v1/page/random/summary"
USER_AGENT = "{Name}/{Version} (Contact: {Author-email})"

def main():
    fields = metadata("random-wikipedia-article")
    headers = {"User-Agent": USER_AGENT.format_map(fields)}

    with httpx.Client(headers=headers, http2=True) as client:
        response = client.get(API_URL, follow_redirects=True)
        response.raise_for_status()
        data = response.json()

    console = Console(width=72, highlight=False)
    console.print(data["title"], style="bold", end="\n\n")
    console.print(data["extract"])
```

You've declared an entry-point script in the `scripts` section in `pyproject.toml`, so users can invoke the application as `random-wikipedia-article`. If you'd like to also allow users to invoke the program with `py -m random_wikipedia_article`, create a `__main__.py` module next to `__init__.py` as shown in [Example 3-3](#).

Managing Dependencies

Let's add the dependencies for `random-wikipedia-article`, starting with Rich, the console output library:

```
$ poetry add rich
Using version ^13.7.1 for rich

Updating dependencies
Resolving dependencies... (0.2s)
```

```
Package operations: 4 installs, 0 updates, 0 removals
```

- Installing mdurl (0.1.2)
- Installing markdown-it-py (3.0.0)
- Installing pygments (2.17.2)
- Installing rich (13.7.1)

```
Writing lock file
```

If you inspect *pyproject.toml* after running this command, you'll find that Poetry has added Rich to the `dependencies` table ([Example 5-5](#)).

Example 5-5. The `dependencies` table after adding Rich

```
[tool.poetry.dependencies]
python = ">=3.10"
rich = "^13.7.1"
```

Poetry also installs the package into an environment for the project. If you already have a virtual environment in *.venv*, Poetry uses that. Otherwise, it creates a virtual environment in a shared location (see [“Managing Environments”](#)).

Caret Constraints

The caret (`^`) is a Poetry-specific extension to version specifiers, borrowed from npm, the package manager for Node.js. *Caret constraints* allow releases with the given minimum version, except those that may contain breaking changes according to the [Semantic Versioning](#) standard. After `1.0.0`, a caret constraint allows patch and minor releases, but no major releases. Before `1.0.0`, only patch releases are allowed—in the `0.*` era, minor releases are allowed to introduce breaking changes.

Caret constraints are similar to *tilde constraints* (see [“Version Specifiers”](#)), but the latter only allow the last version segment to increase. For example, the following constraints are equivalent:

```
rich = "^13.7.1"
rich = ">=13.7.1,<14"
```

On the other hand, tilde constraints typically exclude minor releases:

```
rich = "~13.7.1"
rich = ">=13.7.1,==13.7.*"
```

Caret constraints put an upper bound on the version. As explained in [“Upper Version Bounds in Python”](#), you should avoid upper bounds if you can. Add Rich with only a lower bound instead:

```
$ poetry add "rich>=13.7.1"
```

You can use this command to remove upper bounds from existing caret constraints, as well.² If you specified extras or markers when you first added the dependency, you’ll need to specify them again.

SHOULD YOU CAP DEPENDENCIES?

It's unfortunate that Poetry adds upper bounds to dependencies by default. For libraries, the practice prevents downstream users from receiving fixes and improvements, since constraints aren't scoped to the packages that introduce them, as in Node.js. Many open source projects don't have the resources to backport fixes to past releases. For applications, lock files provide a better way to achieve reliable deployments.

The situation is similar but worse for the Python requirement. Excluding Python 4 by default will cause disruption across the ecosystem when the core Python team eventually releases a new major version. It's unlikely that Python 4 will come anywhere near Python 3 in terms of incompatible changes. Poetry's constraint is contagious in the sense that dependent packages must also introduce it. And it's impossible for Python package installers to satisfy—they can't downgrade the environment to an earlier version of Python.

Whenever possible, replace caret constraints with lower bounds (`>=`), especially for Python itself. After editing *pyproject.toml*, refresh the lock file using the command `poetry lock --no-update`.

Extras and Environment Markers

Let's add the other dependency of `random-wikipedia-article`, the HTTP client library `httpx`. Like in [Chapter 4](#), you'll activate the `http2` extra for HTTP/2 support:

```
$ poetry add "httpx>=0.27.0" --extras=http2
```

Poetry updates the *pyproject.toml* file accordingly:

```
[tool.poetry.dependencies]
python = ">=3.10"
rich = ">=13.7.1"
httpx = {version = ">=0.27.0", extras = ["http2"]}
```


The project requires a recent Python version, so it doesn't require the `importlib-metadata` backport. If you had to support Python versions before 3.8, here's how you'd add the library for those versions:

```
$ poetry add "importlib-metadata>=6.7.0" --python="<3.8"
```

Besides `--python`, the `poetry add` command supports a `--platform` option to restrict dependencies to a specific operating system, such as Windows. This option accepts a platform identifier in the format used by the standard `sys.platform` attribute: `linux`, `darwin`, `win32`. For other environment markers, edit *pyproject.toml* and use the `markers` property in the TOML table for the dependency:

```
[tool.poetry.dependencies]
awesome = {version = ">=1", markers = "implementation_name == 'pypy'"}
```

The Lock File

Poetry records the current version of each dependency in a file named *poetry.lock*, including SHA-256 hashes for its packaging artifacts. If you take a peek inside the file, you'll notice TOML stanzas for `rich` and `httpx`, as well as their direct and indirect dependencies. [Example 5-6](#) shows a simplified version of Rich's lock entry.

Example 5-6. The TOML stanza for Rich in *poetry.lock* (simplified)

```
[[package]]
name = "rich"
version = "13.7.1"
python_versions = ">=3.7.0"
dependencies = {markdown-it-py = ">=2.2.0", pygments = ">=2.13.0,<3.0.0"}
files = [
    {file = "rich-13.7.1-py3-none-any.whl", hash = "sha256:4edbae3..."},
    {file = "rich-13.7.1.tar.gz", hash = "sha256:9be308c..."},
]
```

Use the command `poetry show` to display the locked dependencies in the terminal. Here's what the output looked like after I added Rich:

```
$ poetry show
markdown-it-py 3.0.0 Python port of markdown-it. Markdown parsing, done right!
mdurl          0.1.2 Markdown URL utilities
pygments       2.17.2 Pygments is a syntax highlighting package ...
rich           13.7.1 Render rich text, tables, progress bars, ...
```

You can also display the dependencies as a tree to visualize their relationship:

```
$ poetry show --tree
rich 13.7.1 Render rich text, tables, progress bars, ...
├─ markdown-it-py >=2.2.0
│   └─ mdurl >=0.1,<1.0
└─ pygments >=2.13.0,<3.0.0
```

If you edit *pyproject.toml* yourself, remember to update the lock file to reflect your changes:

```
$ poetry lock --no-update
Resolving dependencies... (0.1s)

Writing lock file
```

Without the `--no-update` option, Poetry upgrades each locked dependency to the latest version covered by its constraint.

You can check if the *poetry.lock* file is consistent with *pyproject.toml*:

```
$ poetry check --lock
```

Resolving dependencies up front lets you deploy applications in a reliable and reproducible manner. It also gives developers in a team a common baseline and makes checks more deterministic—avoiding surprises in

continuous integration (CI). You should commit *poetry.lock* to source control to reap these benefits.

Poetry's lock file is designed to work across operating systems and Python interpreters. Having a single environment-independent, or “universal,” lock file is beneficial if your code must run in diverse environments or if you're an open source maintainer with contributors from all over the world.

By contrast, compiled requirements files quickly become unwieldy. If your project supports Windows, macOS, and Linux on the four most recent feature versions of Python, you'll need to manage a dozen requirements files. Adding another processor architecture or Python implementation only makes things worse.

Universal lock files come at a price, however. Poetry re-resolves dependencies when it installs the packages into an environment. Its lock file is essentially a shrunk world view: it records every package the project might require in a given environment. By contrast, compiled requirements are an exact image of an environment. This makes them more amenable to auditing and more attractive for secure deployments.

Updating Dependencies

You can update all dependencies in the lock file to their latest versions using a single command:

```
$ poetry update
```

You can also provide a specific direct or indirect dependency to update:

```
$ poetry update rich
```

The `poetry update` command doesn't modify the project metadata in *pyproject.toml*. It only updates dependencies within the compatible version range. If you need to update the version range, use `poetry add` with

the new constraint, including any extras and markers. Alternatively, edit *pyproject.toml* and refresh the lock file with `poetry lock --no-update`.

If you no longer need a package for your project, remove it with `poetry remove`:

```
$ poetry remove <package>
```

Managing Environments

Poetry's `add`, `update`, and `remove` commands don't just update dependencies in the *pyproject.toml* and *poetry.lock* files. They also synchronize the project environment with the lock file by installing, updating, or removing packages. Poetry creates the virtual environment for the project on demand.

By default, Poetry stores the environments for all projects in a shared folder. Configure Poetry to keep the environment in a *.venv* directory inside the project instead:

```
$ poetry config virtualenvs.in-project true
```

This setting makes the environment discoverable for other tools in the ecosystem, such as `py` and `uv`. Having the directory in the project is convenient when you need to examine its contents. Although the setting restricts you to a single environment, this limitation is seldom a concern. Tools like Nox and tox are tailor-made for testing across multiple environments (see [Chapter 8](#)).

You can check the location of the current environment using the command `poetry env info --path`. If you want to create a clean slate for your project, use the following commands to remove existing environments and create a new one using the specified Python version:

```
$ poetry env remove --all
$ poetry env use 3.12
```

You can re-run the second command to re-create the environment on a different interpreter. Instead of a version like `3.12`, you can also pass a command like `pypy3` for the PyPy interpreter or a full path like `/usr/bin/python3` for the system Python.

Before you use the environment, you should install the project. Poetry performs editable installs, so the environment reflects any code changes immediately:

```
$ poetry install
```

Enter the project environment by launching a shell session with `poetry shell`. Poetry activates the virtual environment using the activation script for your current shell. With the environment activated, you can run the application from the shell prompt. Just exit the shell session when you're done:

```
$ poetry shell
(random-wikipedia-article-py3.12) $ random-wikipedia-article
(random-wikipedia-article-py3.12) $ exit
```

You can also run the application in your current shell session, using the command `poetry run`:

```
$ poetry run random-wikipedia-article
```

The command is also handy for starting an interactive Python session in the project environment:

```
$ poetry run python
>>> from random_wikipedia_article import main
>>> main()
```

When you run a program with `poetry run`, Poetry activates the virtual environment without launching a shell. This works by adding the environment to the program's `PATH` and `VIRTUAL_ENV` variables (see [“Activation scripts”](#)).

TIP

Just type `py` to get a Python session for your Poetry project on Linux and macOS. This requires the Python Launcher for Unix, and you must configure Poetry to use in-project environments.

Dependency Groups

Poetry allows you to declare development dependencies, organized in dependency groups. Dependency groups aren't part of the project metadata and are invisible to end users. Let's add the dependency groups from [“Development Dependencies”](#):

```
$ poetry add --group=tests pytest pytest-sugar
$ poetry add --group=docs sphinx
```

Poetry adds the dependency groups under the `group` table in *pyproject.toml*:

```
[tool.poetry.group.tests.dependencies]
pytest = "^8.1.1"
pytest-sugar = "^1.0.0"

[tool.poetry.group.docs.dependencies]
sphinx = "^7.2.6"
```

Dependency groups are installed into the project environment by default. You can mark a group as optional using its `optional` key, like this:

```
[tool.poetry.group.docs]
optional = true
```

```
[tool.poetry.group.docs.dependencies]
sphinx = "^7.2.6"
```

WARNING

Don't specify the `--optional` flag when you add a dependency group with `poetry add`—it doesn't mark the group as optional. The option designates optional dependencies that are behind an extra; it has no valid use in the context of dependency groups.

The `poetry install` command has several options that provide finer-grained control over which dependencies are installed into the project environment ([Table 5-3](#)).

Table 5-3. Installing dependencies with `poetry install`

Option	Description
<code>--with=<group></code>	Include a dependency group in the installation.
<code>--without=<group></code>	Exclude a dependency group from the installation.
<code>--only=<group></code>	Exclude all other dependency groups from the installation.
<code>--no-root</code>	Exclude the project itself from the installation.
<code>--only-root</code>	Exclude all dependencies from the installation.
<code>--sync</code>	Remove packages from the environment unless scheduled for installation.

You can specify a single group or multiple groups (separated by commas). The special group `main` refers to packages listed in the `tool.poetry.dependencies` table. Use the option `--only=main` to exclude all development dependencies from an installation. Similarly, the option `--without=main` lets you restrict an installation to development dependencies.

Package Repositories

Poetry lets you upload your packages to the Python Package Index (PyPI) and other package repositories. It also lets you configure the repositories from which you add packages to your project. This section looks at both the publisher and the consumer sides of interacting with package repositories.

NOTE

If you're following along in this section, please don't upload the example project to PyPI. Use the TestPyPI repository instead—it's a playground for testing, learning, and experimentation.

Publishing Packages to Package Repositories

Before you can upload packages to PyPI, you need an account and an API token to authenticate with the repository, as explained in [“Uploading Packages with Twine”](#). Next, add the API token to Poetry:

```
$ poetry config pypi-token.pypi <token>
```

You can create packages for a Poetry project using standard tooling like `build` or with Poetry's command-line interface:

```
$ poetry build
Building random-wikipedia-article (0.1.0)
- Building sdist
- Built random_wikipedia_article-0.1.0.tar.gz
- Building wheel
- Built random_wikipedia_article-0.1.0-py3-none-any.whl
```

Like `build`, Poetry places the packages in the `dist` directory. You can publish the packages in `dist` using `poetry publish`:


```
$ poetry publish
```

You can also collapse the two commands into one:

```
$ poetry publish --build
```

Let's upload the example project to TestPyPI, a separate instance of the Python Package Index for testing distributions. If you want to upload packages to a repository other than PyPI, you need to add the repository to your Poetry configuration:

```
$ poetry config repositories.testpypi https://test.pypi.org/legacy/
```

First, create an account and an API token on TestPyPI. Next, configure Poetry to use that token when uploading to TestPyPI:

```
$ poetry config pypi-token.testpypi <token>
```

You can now specify the repository when publishing your project. Feel free to try this with your own version of the example project:

```
$ poetry publish --repository=testpypi
Publishing random-wikipedia-article (0.1.0) to TestPyPI
- Uploading random_wikipedia_article-0.1.0-py3-none-any.whl 100%
- Uploading random_wikipedia_article-0.1.0.tar.gz 100%
```

Some package repositories use HTTP basic authentication with a username and password. You can configure the credentials for such a repository like this:

```
$ poetry config http-basic.<repo> <username>
```

The command prompts you for the password and stores it in the system keyring, if available, or in the *auth.toml* file on disk.

Alternatively, you can also configure a repository via environment variables (replace `<REPO>` with the repository name in uppercase, such as `PYPI`):

```
$ export POETRY_REPOSITORIES_<REPO>_URL=<url>
$ export POETRY_PYPI_TOKEN_<REPO>=<token>
$ export POETRY_HTTP_BASIC_<REPO>_USERNAME=<username>
$ export POETRY_HTTP_BASIC_<REPO>_PASSWORD=<password>
```

Poetry also supports repositories that are secured by mutual TLS or use a custom certificate authority; see the [official documentation](#) for details.

Fetching Packages from Package Sources

You’ve seen how to upload your package to repositories other than PyPI. Poetry also supports alternate repositories on the consumer side: you can add packages to your project from sources other than PyPI. While upload targets are a user setting and stored in the Poetry configuration, package sources are a project setting, stored in *pyproject.toml*.

Add a package source using the command `poetry source add`:

```
$ poetry source add <repo> <url> --priority=supplemental
```

Poetry searches supplemental sources if the package wasn’t found on PyPI. If you want to disable PyPI, configure a primary source instead (the default priority):

```
$ poetry source add <repo> <url>
```

You configure credentials for package sources just like you do for repositories:

```
$ poetry config http-basic.<repo> <username>
```

You can now add packages from the alternate source:

```
$ poetry add httpx --source=<repo>
```

The following command lists the package sources for the project:

```
$ poetry source show
```

WARNING

Specify the source when adding packages from supplemental sources. Otherwise, Poetry searches all sources when looking up a package. An attacker could upload a malicious package to PyPI with the same name as your internal package (*dependency confusion attack*).

Extending Poetry with Plugins

Poetry comes with a plugin system that lets you extend its functionality. Use `pipx` to inject the plugin into Poetry's environment:

```
$ pipx inject poetry <plugin>
```

Replace `<plugin>` with the name of the plugin on PyPI.

If the plugin affects the build stage of your project, add it to the build dependencies in `pyproject.toml`, as well. See [“The Dynamic Versioning Plugin”](#) for an example.

By default, `pipx` upgrades applications without the injected packages. Use the option `--include-injected` to also upgrade application plugins:

```
$ pipx upgrade --include-injected poetry
```

If you no longer need the plugin, remove it from the injected packages:

```
$ pipx uninject poetry <plugin>
```

If you're no longer sure which plugins you have installed, list them like this:

```
$ poetry self show plugins
```

In this section, I'll introduce you to three useful plugins for Poetry:

```
poetry-plugin-export
```

Lets you generate requirements and constraints files

```
poetry-plugin-bundle
```

Lets you deploy the project to a virtual environment

```
poetry-dynamic-versioning
```

Populates the project version from the VCS

Generating Requirements Files with the Export Plugin

Poetry's lock file is great to ensure that everybody on your team, and every deployment environment, ends up with the same dependencies. But what do you do if you can't use Poetry in some context? For example, you may need to deploy your project on a system that has only a Python interpreter and the bundled pip.

As of this writing, there's no lock file standard in the wider Python world; each packaging tool that supports lock files implements its own format.³ None of these lock file formats has support in pip. But we do have requirements files.

Requirements files let you pin packages to an exact version, require their artifacts to match cryptographic hashes, and use environment markers to restrict packages to specific Python versions and platforms. Wouldn't it be nice if you could generate one from your *poetry.lock* for interoperability with non-Poetry environments? This is precisely what the export plugin achieves.

Install the plugin with pipx:

```
$ pipx inject poetry poetry-plugin-export
```

The plugin powers the `poetry export` command, which sports a `--format` option to specify the output format. By default, the command writes to the standard output stream; use the `--output` option to specify a destination file:

```
$ poetry export --format=requirements.txt --output=requirements.txt
```

Distribute the requirements file to the target system and use pip to install the dependencies (typically followed by installing a wheel of your project):

```
$ python3 -m pip install -r requirements.txt
```

Exporting to requirements format is useful beyond deploying. Many tools work with requirements files as the de facto industry standard. For example, you can scan a requirements file for dependencies with known security vulnerabilities using a tool like [Safety](#).

Deploying Environments with the Bundle Plugin

In the previous section, you saw how to deploy your project on a system without Poetry. If you do have Poetry available, you might be wondering: can you just deploy with `poetry install`? You could, but Poetry performs an editable install of your project—you'll be running your application from the source tree. That may not be acceptable in a production environment. Editable installs also limit your ability to ship the virtual environment to another destination.

The bundle plugin allows you to deploy your project and locked dependencies to a virtual environment of your choosing. It creates the environment, installs the dependencies from the lock file, then builds and installs a wheel of your project.

Install the plugin with pipx:

```
$ pipx inject poetry poetry-plugin-bundle
```

After installation, you'll see a new `poetry bundle` subcommand. Let's use that to bundle the project into a virtual environment in a directory named `app`. Use the `--python` option to specify the interpreter for the environment and the option `--only=main` to exclude development dependencies:

```
$ poetry bundle venv --python=/usr/bin/python3 --only=main app  
- Bundled random-wikipedia-article (0.1.0) into app
```

Test the environment by running the entry-point script for the application:⁴

```
$ app/bin/random-wikipedia-article
```

You can use the bundle plugin to create a minimal Docker image for production. Docker supports *multistage builds*, where the first stage builds the application in a full-fledged build environment, and the second stage copies the build artifacts over into a minimal runtime environment. This allows you to ship slim images to production, speeding up deployments and reducing bloat and potential vulnerabilities in your production environments.

In [Example 5-7](#), the first stage installs Poetry and the bundle plugin, copies the Poetry project, and bundles it into a self-contained virtual environment. The second stage copies the virtual environment into a minimal Python image.

Example 5-7. Multistage Dockerfile with Poetry

```
FROM debian:12-slim AS builder ❶  
RUN apt-get update && \
```

```

    apt-get install --no-install-suggests --no-install-recommends --yes pipx
ENV PATH="/root/.local/bin:${PATH}"
RUN pipx install poetry
RUN pipx inject poetry poetry-plugin-bundle
WORKDIR /src
COPY . .
RUN poetry bundle venv --python=/usr/bin/python3 --only=main /venv

FROM gcr.io/distroless/python3-debian12 ❷
COPY --from=builder /venv /venv ❸
ENTRYPOINT ["/venv/bin/random-wikipedia-article"] ❹

```

- ❶ The first `FROM` directive introduces the build stage, where you build and install your project. The base image is a slim variant of the Debian stable release.
- ❷ The second `FROM` directive defines the image that you deploy to production. The base image is a *distroless* Python image for Debian stable: Python language support minus the operating system.
- ❸ The `COPY` directive allows you to copy the virtual environment over from the build stage.
- ❹ The `ENTRYPOINT` directive lets you run the entry-point script when users invoke `docker run` with the image.

If you have Docker installed, you can try this out. First, create a *Dockerfile* in your project with the contents from [Example 5-7](#). Next, build and run the Docker image:

```

$ docker build -t random-wikipedia-article .
$ docker run --rm -ti random-wikipedia-article

```

You should see the output from `random-wikipedia-article` in your terminal.

The Dynamic Versioning Plugin

The dynamic versioning plugin populates the version in the project meta-data from a Git tag. Keeping the version in a single place reduces churn (see [“Single-Sourcing the Project Version”](#)). The plugin is based on Dunamai, a Python library for deriving standards-compliant version strings from tags in your version control system.

Install the plugin with `pipx` and enable it for your project:

```
$ pipx inject poetry "poetry-dynamic-versioning[plugin]"
$ poetry dynamic-versioning enable
```

The second step enables the plugin in the `tool` section of `pyproject.toml`:

```
[tool.poetry-dynamic-versioning]
enable = true
```

Remember that you have installed the Poetry plugin globally. The explicit opt-in ensures that you don't accidentally start overwriting the version field in unrelated Poetry projects.

Build frontends like `pip` and `build` need the plugin when they build your project. For this reason, enabling the plugin also adds it to the build dependencies in `pyproject.toml`. The plugin brings its own build backend, which wraps the one provided by Poetry:

```
[build-system]
requires = ["poetry-core>=1.0.0", "poetry-dynamic-versioning>=1.0.0,<2.0.0"]
build-backend = "poetry_dynamic_versioning.backend"
```

Poetry still requires the `version` field in its own section. Set the field to `"0.0.0"` to indicate that it's unused:

```
[tool.poetry]
version = "0.0.0"
```


You can now add a Git tag to set your project version:

```
$ git tag v1.0.0
$ poetry build
Building random-wikipedia-article (1.0.0)
- Building sdist
- Built random_wikipedia_article-1.0.0.tar.gz
- Building wheel
- Built random_wikipedia_article-1.0.0-py3-none-any.whl
```

The plugin also replaces `__version__` attributes in Python modules. This mostly works out of the box, but you need to declare `src` layout if you use it:

```
[tool.poetry-dynamic-versioning]
enable = true
substitution.folders = [{path = "src"}]
```

Let's add a `--version` option to the application. Edit `__init__.py` in the package to add the following lines:

```
import argparse

__version__ = "0.0.0"

def main():
    parser = argparse.ArgumentParser(prog="random-wikipedia-article")
    parser.add_argument(
        "--version", action="version", version=f"%(prog)s {__version__}"
    )
    parser.parse_args()
    ...
```

Before proceeding, commit your changes, but without adding another Git tag. Let's try the option in a fresh installation of the project:

```
$ uv venv
$ uv pip install .
```

```
$ py -m random_wikipedia_article --version
random-wikipedia-article 1.0.0.post1.dev0+51c266e
```

As you can see, the plugin rewrote the `__version__` attribute during the build. Since you didn't tag the commit, Dunamai marked the version as a developmental post-release of `1.0.0` and appended the commit hash using a local version identifier.

Summary

Poetry provides a unified workflow to manage packaging, dependencies, and environments. Poetry projects are interoperable with standard tooling: you can build them with `build` and upload them to PyPI with Twine. But the Poetry command-line interface also provides convenient short-hands for these tasks and many more.

Poetry records the precise working set of packages in its lock file, giving you deterministic deployments and checks, as well as a consistent experience when collaborating with others. Poetry can track development dependencies; it organizes them in dependency groups that you can install separately or together. You can extend Poetry with plugins—for example, to deploy the project into a virtual environment or to derive the version number from Git.

If you need reproducible deployments for an application, if your team develops on multiple operating systems, or if you just feel that standard tooling adds too much overhead to your workflows, you should give Poetry a try.

¹ Sébastien Eustace, [“Support for PEP 621”](#), November 6, 2020.

² The command also keeps your lock file and project environment up-to-date. If you edit the constraint in *pyproject.toml*, you'll need to do this yourself. Read on to learn more about lock files and environments.

3 Apart from Poetry's own *poetry.lock* and the closely related PDM lock file format, there's pipenv's *Pipfile.lock* and the `conda-lock` format for Conda environments.

4 Replace *bin* with *Scripts* if you're on Windows.